COMP9001
Lecture 4
Introduction to Programming Containers

Presenter: Dr Armin Chitizadeh

Version 0.1

THE UNIVERSITY OF SYDNEY

Inspired by Hazem El-Alfy 2024 slides

CRICOS 00026A TEQSA PRV12057

We recognise and pay respect to the Elders and communities – past, present, and emerging – of the lands that the University of Sydney's campuses stand on. For thousands of years they have shared and exchanged knowledges across innumerable generations for the benefit of all.

THE UNIVERSITY OF SYDNEY

Consultations

- After Lecture
- Monday 12pm – 1pm Online
- Thursdays 5:30pm to 6:30pm (Hybrid)
- And more in the future!

Details on ed,
(There will be a page about it to be better organised)

✏️ Assessments

✓ Homework 1 (5 Seconds of Summer)

Due: Sun March 22nd, 11:59pm

Please remember about our limitations on answering the forum

Expect 2 business days,
only during working hours,
no weekend pr public-holidays answering

Mid-semester Review Intense Workshop

- Friday of mid-semester break (More info to be announced soon!)
  - Details might change

Assessment Schedule

Assessment Schedule

Task	Description	Release Week	Due Week	Weighting
Homework 1	Coding assessment to test student understanding of topics covered.	Week 1	Week 3	5%
Assignment 1	Coding assessment to test student understanding of topics covered. Focused on introductory topics.	Week 5	Week 7	15%
Assignment 2	Coding assessment to test student understanding of topics covered. Focused on more challenging topics in the unit.	Week 8	Week 11	25%
In-Lab-Checkpoint	At the end of each lab, students are asked to do a simple quiz. The primary purpose of these assessment is to ensure that students understand the topic thoroughly before concluding the tutorial. The highest 8 marks out of a total of 12 will be used to calculate the mark.	N/A	Weekly post-lab	5%

“Yay” moment!

Photo by Jonathan Borba on Unsplash

”Yay” moment!

https://www.menti.com/alxohsrkkw1g

Where are we in the unit?

Week 1 Intro

Week 2 Variables

Week 3 Conditionals

Week 4 containers

Week 5 Flow/Functions

Week 6 Iterations

Week 7 Classes/Objects

Week 8 File I/O

Week 9 Adv Flow

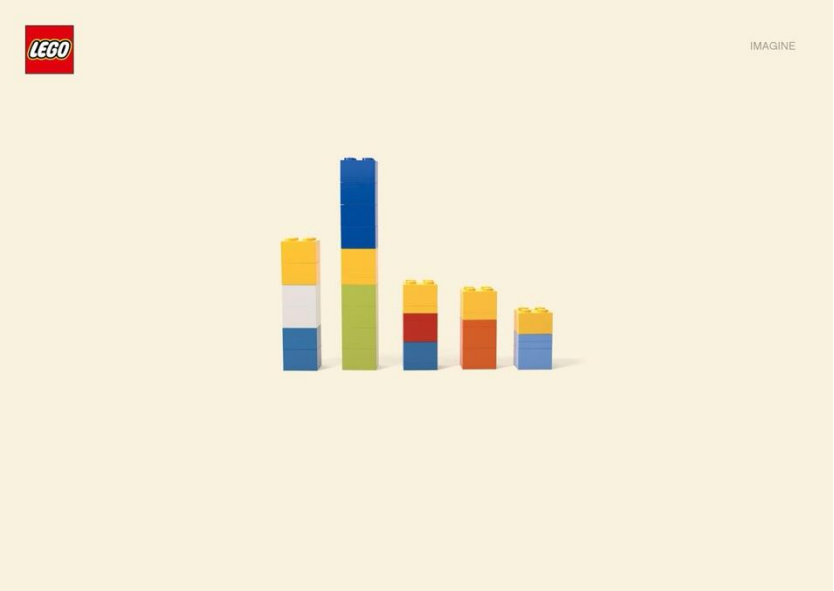
Week 10 Testing

Week 11 Recursion

Week 12 Multi Arrays

Week 13 Revision

Final Project



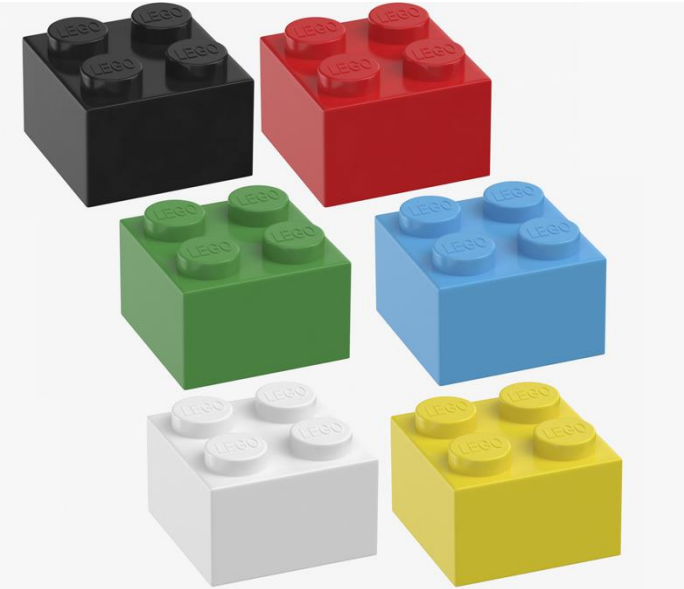
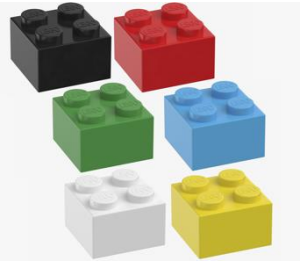


Image credit:
https://www.adsoftheworld.com/media/print/lego_the_simpsons
https://www.turbosquid.com/3d-models/lego-bricks-2x2-3d-model-1491947

Data Types

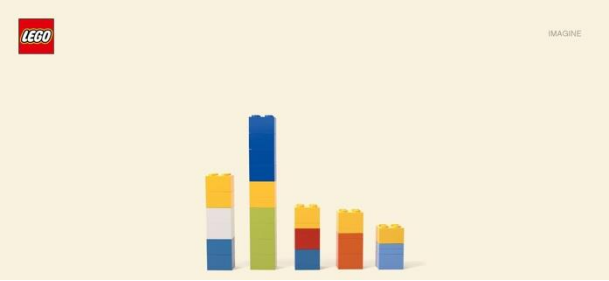
<u>Basic</u> (Single Value)

- Integers
- Floating Points
- Boolean
- Characters



<u>Containers</u> (Multiple Values)

- Strings
- Lists
- Tuples
- Dictionaries



Esfandiar's Seven Labours

Artist: Skhokoufeh Hassanzarei
Link: https://artrepreneur.com/p/2YSHw7uu33PQjdWj7/art/ctmb96YKqFYW6iT7W

Esfandiar's Seven Labours

- How do we store these:
    1. Two monstrous wolves
    2. Two man eating lions
    3. Dragon
    4. Evil Magic
    5. Simurgh
    6. Storm
    7. Desert

esfandiar_seven_labour_bad.py

```python
# Storing Esfandiar's Seven Labours
# Not the best way

battle1 = "wolves"
battle2 = "lions"
battle3 = "dragon"
battle4 = "evil_magic"
battle5 = "simurgh"
battle6 = "storm"
battle7 = "desert"

print(battle1, battle2, battle3,
    battle4, battle5, battle6, battle7)
```

list

List – What and Why?

- A *data structure* is a way to organize data that we wish to process with a computer program.

- A list is a *contiguous* block of memory containing multiple values not all of the same type.

- List solves the problem of having many variables.

Python List

- A multi-element list is defined as comma separated objects within the square brackets [and]

- Elements of the same type = homogenous list
  Elements of different types = heterogenous list

- Python's list is <u>NOT</u> the same as arrays, i.e.

  arrays and lists are **not** the same in Python!

Esfanidar's Seven Labour List

esfandiar_seven_labour.py

```python
# Storing Esfandiar's Seven Labours
# A better way
battle_list = ["wolves", "lions", "dragon",
                        "evil_magic", "simurgh", "storm",
                        "desert"]

print(battle_list)
```

Initialisation

- Many ways to initialize lists.
- Can you spot examples of each initialization below?

```
# Initialize lists
empty_ls = []
tutors = ["Avinash", "Joshua", "Luis"]
x = [5, 10, 15, 20, 25]
help_ls = list("help")
```

Python List

- How to access them individually?

```
battle_list[0]
```

- How to measure length of a list?

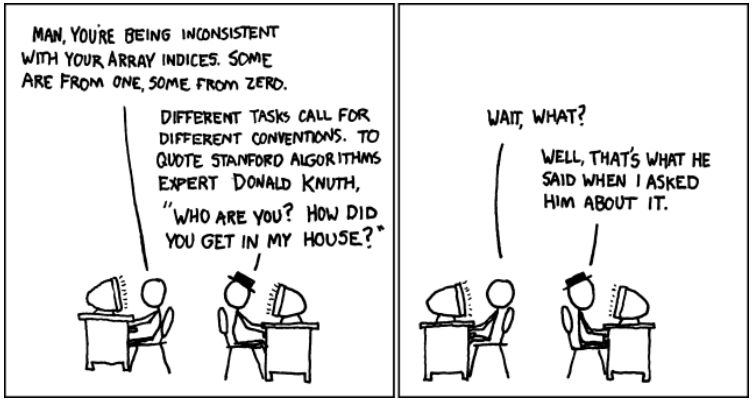
```
len(battle_list)
```

- How to update them?

```
battle_list[1] = "beast_akvan"
```

Zero-based Indexing and One-based Indexing

- There are programming languages that use one-based indexing. The first element starts at one instead of zero.
- Consequences:
    - Index-1: Index of last element is the **<u>same</u>** as total number of elements.
    - Index-0: Index of last element is **<u>one less than</u>** the total number of elements.



- Python is Zero-based

Rostam's Seven Labours

"I want to record everything as I go through"

- Rostam

- How do we store these once they happen:
    1. Lion
    2. Thirst
    3. Dragon
    4. Temptress
    5. Beasts
    6. Rescue the King
    7. White Beast

Artist:
Saeed Ramez
https://au.pinterest.com/pin/412360909638705440/

Playing with Lists in Python!

List Manipulation

Method	Description
`append(x)`	Adds an element **`x`** to the end of the list.
`count(x)`	Returns the number of times element **`x`** appears in the list.
`index(x)`	Returns the index of the first occurrence of element **`x`** in the list.
`insert(index, x)`	Inserts an element **`x`** at a given **`index`**. Note that the first element in the list has index **`0`**.
`pop(index)`	Removes the element at the given index and return it. The parameter index is optional. If it is not specified, **`list.pop()`** removes and returns the last element in the list.
`remove(x)`	Removes the first occurrence of element **`x`** from the list.
`reverse(x)`	Reverses the elements in the list.
`sort()`	Sorts the elements in the list in an ascending order.

Liang, Y, et al. *Introduction to Python Programming and Data Structures, Global Edition*, Pearson Education, Limited, 2022. *ProQuest Ebook Central,*

List Manipulation

- Rostam Python Demo!

list1 + list2

```
[1,2,3] + [4,5,6] =  [1,2,3,4,5,6]
```

Slicing in Python

Slicing

- The slicing operator returns a slice of the given list using the syntax **`list[start:end]`**
  - The `start` index is the start of the slice (**inclusive**)
  - The `end` index is the end of the slice (**exclusive**)
- For `alphabet = ['a', 'b', 'c', 'd', 'e', 'f']` try:
  - `alphabet[2:5]`
  - `alphabet[2:]`
  - `alphabet[:5]`
  - `alphabet[-1]`
  - `alphabet[-3]`
  - `alphabet[-7]`

Practice Slicing on Ed Lesson!

Examples by Joshua Blair and Ehab Ahmed

Slicing with step

- The slicing operator returns a slice of the given list using the syntax **`list[start:end:step]`**
  - The `start` index is the start of the slice (**inclusive**)
  - The `end` index is the end of the slice (**exclusive**)
  - The `step` is interval between elements to include in a slice
- For `alphabet = ['a', 'b', 'c', 'd', 'e', 'f']` try:
  - `alphabet[0:6:2]`
  - `alphabet[::2]`
  - `alphabet[::3]`
  - `alphabet[::-1]`

Two Dimensional lists

- Lists can be two dimensional!

Two Dimensional Lists

```
game = [['x', '-', 'o'], ['-', 'x', 'o'], ['-', '-', 'o']]
```

```
game = [
    ['x', '-', 'o'],
    ['-', 'x', 'o'],
    ['-', '-', 'o']
]
```

Accessing

- Try:
    - `print(game)`
    - `print(game[1])`
    - `print(game[1][2])`

Tuples

Tuples

- You can use tuples for storing a fixed list of elements
- Tuples are like lists, but their elements are fixed
- Once tuple is created, you cannot add , delete, or replace any element **accidentally**

- A multi-element tuple is defined as comma separated objects within the parentheses `(` and `)`
- You can do the same operations you did on `list` on `tuples` too?

Try the ed lesson yourself!

Mutable and Immutable

- List and tuples have common operations:
  - Length using `len(empty_ls)`
  - Accessing `i`th item of s from origin `0` e.g. `tutors[1]`
  - Slices from index `i` to `j-1` e.g. `tutors[1:2]`

- Lists are **mutable**. Tuples are **immutable**.
  - You can modify lists' elements after creation.
  - Like string objects, once you create a tuple, you cannot modify its elements.
  - Tuples are used to pass multiple values out a function to reduce the risk of modifying the function's return values.

Deep Dive Into How Lists and Tuples Are Implemented in Python

- You take the **blue pill** – the story ends, you wake up in your bed and believe whatever you want to believe.

- You take the **red pill** – you stay in Wonderland, and I show you how deep the rabbit hole goes.

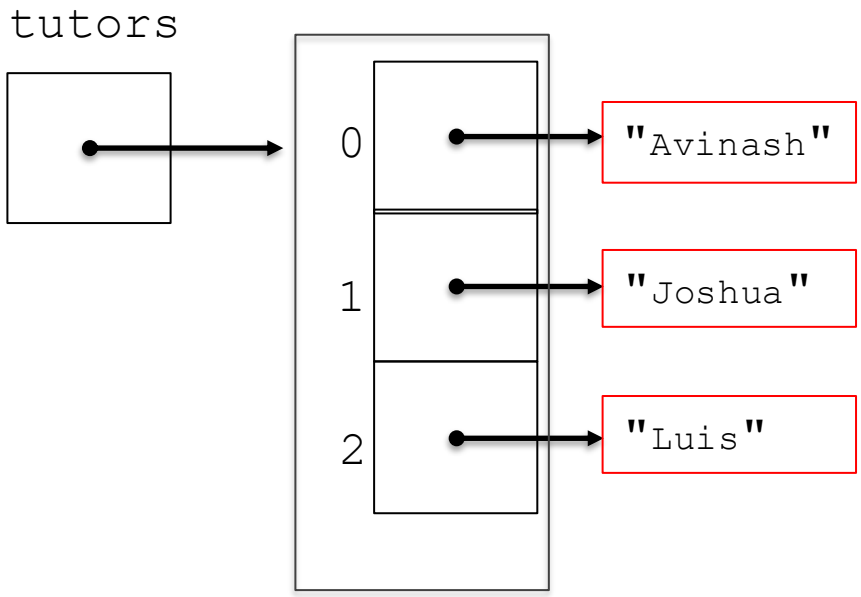
Vote:
Deep Dive Into How Lists and Tuples Are Implemented in Python

- You take the **blue pill** – the story ends, you wake up in your bed and believe whatever you want to believe.

- You take the **red pill** – you stay in Wonderland, and I show you how deep the rabbit hole goes

Object Level Trace Diagram

```
1 # Initialize lists
2 empty_ls = []
3 tutors = ["Avinash", "Joshua", "Luis"]
4 ls_one_element = [5]
```



String, List, Tuples

```python
greet = 'hello'
print(greet[0])
print(greet[1])
print(greet[2])
print(greet[3])
print(greet[4])
# End program
```

String, List, Tuples

- Can you convert a string to list and tuple?

- Does this work:

```
"Hello"[0]
```

Menti!

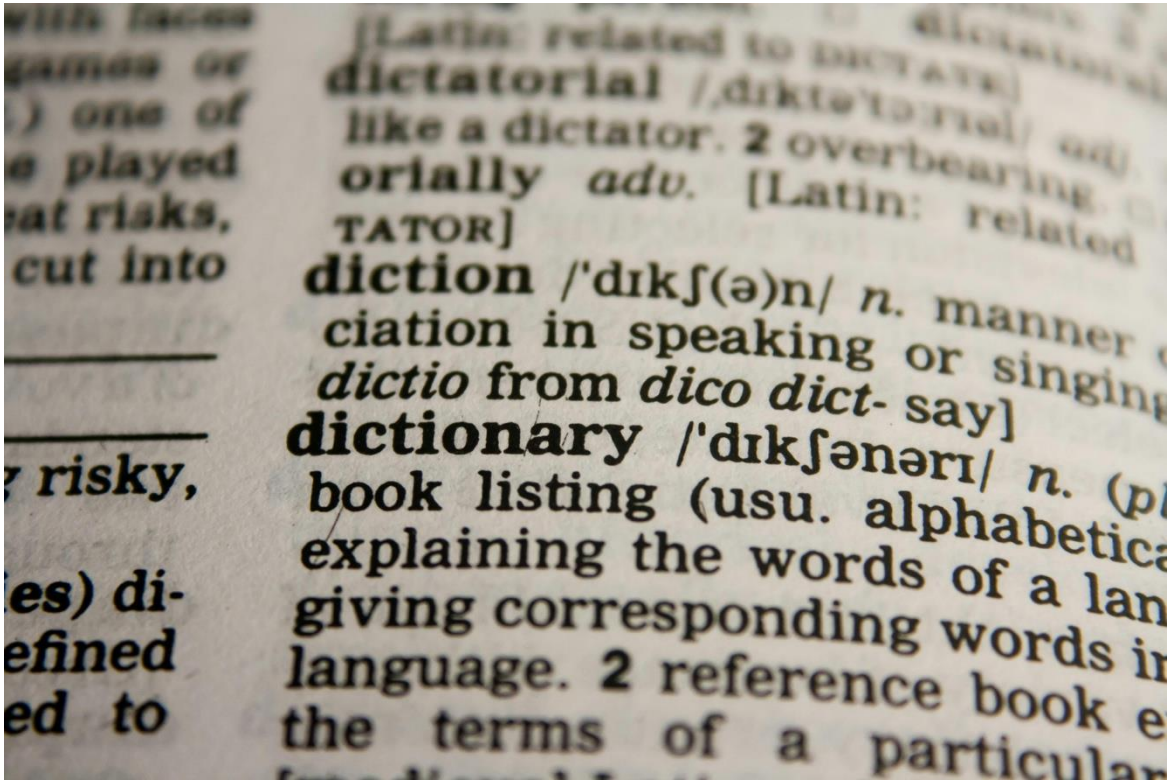
https://www.menti.com/al8qg1f2vuem

Rastak- Sornaye Nowruz

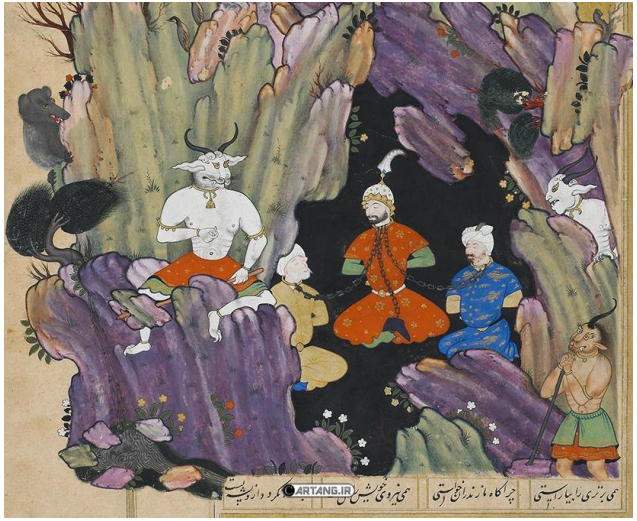
Break

https://www.youtube.com/watch?v=3IOMAXbwBI4

Dictionaries



How can we be better organised?



Dictionary in Python

- Dictionary in Python are used to store data values in key:value pairs.
- A dictionary is a collection which is ordered (in Python version 3.7), changeable and do not allow duplicates.
- Dictionary items are presented in key:value pairs and can be referred to by using the key name.
- Python dictionaries allow us to associate a value to a unique key, and then to quickly access this value.
- Dictionaries are changeable, meaning that we can change, add or remove items after the dictionary has been created.

Dictionary in Practice

- We are creating dictionary using curly braces or dict() method

```
marks_hw = {
    "0646": 89,
    "2134": 95,
    "2342": 93
}
# or
marks_exam = dict()
marks_exam["0646"] = 34
marks_exam["2134"] = 93
marks_exam["9134"] = 100
marks_exam["4134"] = 99
```

Can We Make Battles of Heroes More Organised?

Personal Dictionary

- Go to Ed on this link and create your own dictionary with any details you want. Show your results to the person next to you.

Python Containers (quick revision)

- Python collections can be list, tuples, dictionary
- Fundamental operations:
    - Initialize
    - Access
    - Modify elements (if possible)

Command Line Arguments

```
achi0646@AHH6CFCMWFJ wk04 % python3 battles.py rostam
```

Command Line Arguments

- Inputs are passed in through the command line instead of using input() function.

- Command line argument are stored in a variable (an array of string objects) called `argv` in module `sys`.

- First element will always be the script name.

Summary for part1

- Lists are contiguous blocks of memory.

- We can easily access (and manipulate) a group of related data.

- You can have lists of anything. As the programmer, you are deciding what memory you need to have and then how it is processed.

- You can mix different type of objects into the same list in Python but this may not work in the other languages. It is best that you use the *same type of objects* in your list.

Summary

- Another way of getting inputs from users – command line arguments which is stored in `sys.argv`.

- The first command line argument is always the script name.

- Dictionary items are presented in key:value pairs and can be referred to by using the key name.

Fully anonymous feedback form for COMP9001 students – Fill out form